
Тезис доклада (держат в голове всё время)

Цель — изучить возможный стек и подходы при crawling и aggregating данных. **Гипотеза:** существует ограниченный набор приёмов (strategies), применимый к произвольному множеству сайтов. **Исследование** — на примерах популярных сайтов, каждый — с реальной структурной проблемой. **Результат** — показать, что ограниченный набор стратегий закрывает основные случаи, а дальше данные проходят через единый маппинг → upsert → индексацию.

Каждый блок ниже явно привязан к одному из 4 пунктов этого тезиса.

Intro (Слайд 1)

Это Job-Crawler-Demo — не job board, а демонстрация инфраструктуры, которая за job board стоит: crawling разнородных, местами враждебных к ботам сайтов → normalization того, что они реально отдадут → индексация для полноценного faceted search. Та же форма задачи, что в любом production data-ingestion pipeline.

Идея доклада простая: изучить, какой стек и какие подходы реально нужны для crawling и aggregating данных. Рабочая гипотеза — что существует ограниченный набор приёмов, который закрывает crawling практически любого сайта. Проверяем это на четырёх популярных источниках.

Stack (Слайд 2)

Один demo, но задействован весь стек, который встречается в реальных системах: Node.js/Express/ TypeScript на backend, Next.js с Feature-Sliced Design на frontend, Puppeteer и Axios+Cheerio для crawling, Postgres/Redis/Elasticsearch для хранения, JWT для auth. AI enrichment через Claude API — в планах, сейчас интерфейс уже заложен через MockAIEnricher.

Гипотеза + запуск live crawl (Слайд 3 → /sources)

Вот что происходит при краулинге источника: сайт → CrawlStrategy → RawVacancy → upsert в Elasticsearch → search UI. Два выделенных этапа — это как раз то место, где применяется “маппинг”, но в двух разных смыслах. Вернёмся к этому чуть позже.

Гипотеза, которую проверяем: один и тот же интерфейс CrawlStrategy должен уметь закрыть произвольно разные сайты — без изменения самого интерфейса.

Переключаемся в браузер, /sources :

Вот 4 источника — Habr Career, RemoteOK, WeWorkRemotely, Craigslist. У каждого — своя CrawlStrategy. Запущу прогон прямо сейчас на RemoteOK — он самый быстрый: у него нет

отдельного detail-page прохода, только листинг.

Действие: кликаем на RemoteOK → **Run crawl**. Статус переходит в **RUNNING**, начинают идти Execution logs.

Пока он крутится в фоне — за минуту-полторы — пройдемся по тому, что вообще отличает эти четыре сайта друг от друга.

Разные структуры данных (Слайд 4, краулинг идёт в фоне)

Это второй пункт гипотезы: данные на разных сайтах лежат в совершенно разных структурах — и набор приёмов должен закрывать их все.

— **Habr Career** — полностью серверный рендеринг, но с двухуровневым доступом: сначала listing (стандартный paging), потом отдельный HTTP-запрос на detail-страницу каждой вакансии за дополнительными полями — там уже структурированный `schema.org/JobPosting` JSON-LD, что стабильнее, чем парсить CSS-классы.

— **RemoteOK** — тот, что крутится сейчас — прикрыт Cloudflare, обычный HTTP-запрос получает 403. Здесь нужен Puppeteer с реальным browser fingerprint. Зато вся информация уже есть в JSON-LD прямо в листинге — detail-запрос не нужен вообще.

— **WeWorkRemotely** — здесь листинг тоже через headless Puppeteer, но после первого же перехода на detail-страницу браузер словили по fingerprint. Решение — тот же контент отдаёт RSS-фид той же категории, без Puppeteer вообще, обычным Axios.

— **Craigslist** — никакой защиты от ботов нет, но это generic classifieds, не только tech. Ограничиваем через `cat=sof` категорию, плюс fan-out по нескольким городам.

То есть один и тот же интерфейс `CrawlStrategy` закрывает: static HTML, Cloudflare-gated JS, RSS, и multi-city fan-out — без изменения самого интерфейса.

Результат live crawl

Действие: переключаемся на вкладку RemoteOK — статус должен быть **COMPLETED**, видно число собранных вакансий.

Прогон завершился — вот собранные вакансии. Гипотеза подтверждена на практике: за несколько секунд прогона мы получили реальные данные с сайта, защищённого от ботов, тем же самым абстрактным интерфейсом, что и обычный static HTML.

Маппинг / унификация (Слайд 5)

Третий пункт — унификация данных. Каждый сайт отдаёт свою собственную структуру полей — привести их к единой целевой модели `RawVacancy` — это и есть mapping, в ETL-смысле этого слова: например, `hiringOrganization.name` из JSON-LD у Craigslist превращается в наше поле `company`, а `jobLocationType === "TELECOMMUTE"` у Habr — в `isRemote`. Это происходит внутри каждой стратегии, ещё до того, как что-либо попадёт в базу.

Здесь же стоит развести два значения слова “mapping” — потому что второе значение, mapping в смысле Elasticsearch (схема индекса, типы полей), это отдельная, уже не ETL-история — к ней переходим сейчас.

Upsert + Elasticsearch (Слайд 6 + “View raw ES data”)

Вот что реально пишется в Elasticsearch одним upsert’ом. Дедупликация — по ключу `sourceId:externalId`, так что повторный краулинг того же источника не плодит дубли, а просто обновляет `lastSeenAt`. Здесь же — Elasticsearch mapping в буквальном смысле: типы полей, `keyword` для фасетов и агрегаций, `text` для полнотекстового поиска.

Отдельный момент — версионирование схемы: `CRAWLER_RESULTS_SCHEMA_VERSION`. Поскольку Elasticsearch здесь — производный индекс, а не источник истины, несовместимое изменение схемы не мигрируется на месте — индекс просто удаляется и пересоздаётся пустым, а данные возвращаются обратно только после повторного краулинга источников (вручную или “crawl all”), поскольку сами вакансии нигде, кроме ES, не хранятся.

Действие (если время позволяет): возвращаемся на вкладку Sources, у любой вакансии нажимаем “View raw ES data” — показываем тот самый JSON-документ вживую.

Вот этот же документ — можно посмотреть его прямо в UI, это ровно то, что реально лежит в индексе.

Search live-демо (/search)

Действие: переключаемся на заранее подготовленную вкладку `/search`.

А вот итог всей цепочки — конечный пользовательский результат. Поиск по всему собранному корпусу, не по одному источнику.

Действие: вводим свободный текст (например `python`), показываем результаты.

Свободный текст ищет по title, company, description сразу по всем источникам.

Действие: включаем 1–2 facet-чекбоксы (например Remote + Seniority).

Facets — Specialization, Seniority, Remote/On-site, Location, Company — с живыми счётчиками по текущей выборке.

Действие: начинаем печатать в поле поиска (например `Je`), показываем autocomplete-подсказки.

И autocomplete — подсказки по title/company, начинают работать от двух символов.

Итог (Слайд 7)

Что демо реально доказывает:

Первое — один и тот же абстрактный интерфейс `CrawlStrategy` не менялся ни разу на четырёх структурно разных сайтах — гипотеза подтвердилась.

Второе — реальные anti-bot проблемы получили реальные решения прямо по ходу разработки, не постфактум придуманные.

Третье — поисковый слой ведёт себя как production: фасеты, автокомплит, пересобираемый индекс.

Четвёртое — вежливость by design: rate-limit с jitter и кэш на каждый источник, потому что демо-скрапер — всё равно скрапер.

Закрытие

На этом всё — стек и подходы, которые мы хотели изучить, подтвердили рабочую гипотезу на практике. Готов ответить на вопросы.